

ANEXO I

REPORTE DE ESTANCIAS DE OCTUBRE

NOMBRE PROYECTO: Sistema de Gestión de Servicios Operativa de Mantenimiento de Habitaciones y Espacios Comunes SGSOM (Backend)

NOMBRE: Juan Leonardo Cruz Flores

MATRÍCULA: 202300097

PRÁCTICA PROFESIONAL: ☒ EI ☐ EII ☐ ESTADÍA ☐ SS ☐ ESC. DE PRÁCTICA

REPORTE NO. INTERVALO DE FECHAS:

A. DESCRIPCIÓN DE ACTIVIDADES REALIZADAS

Después de conversar la problemática con mi asesor empresarial, hallar los requerimientos, encontrar las problemáticas y flujos de uso, pude elaborar los distintos diagramas que representan la estructura, uso y herramientas del proyecto que se elaborará.

Comencé con los UML:

Diagrama de Casos de Uso:

- Actores identificados: Personal de Mantenimiento, Supervisor, Administrador del Sistema
- Casos de uso principales: Gestionar Edificios, Gestionar Cuartos, Registrar Mantenimientos
- Casos de uso secundarios: Cambiar Estado de Cuarto, Emitir Alertas, Consultar Historial
- Relaciones: Include, Extend, Generalization

Diagrama de Casos de Uso - SGSOM (JW Mantto)

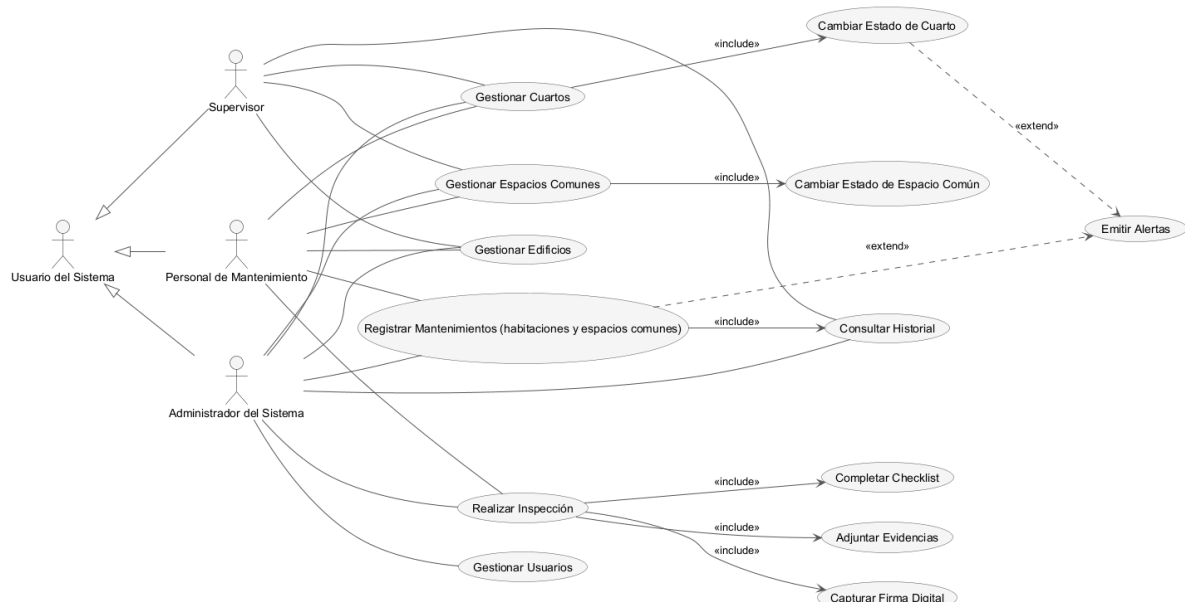
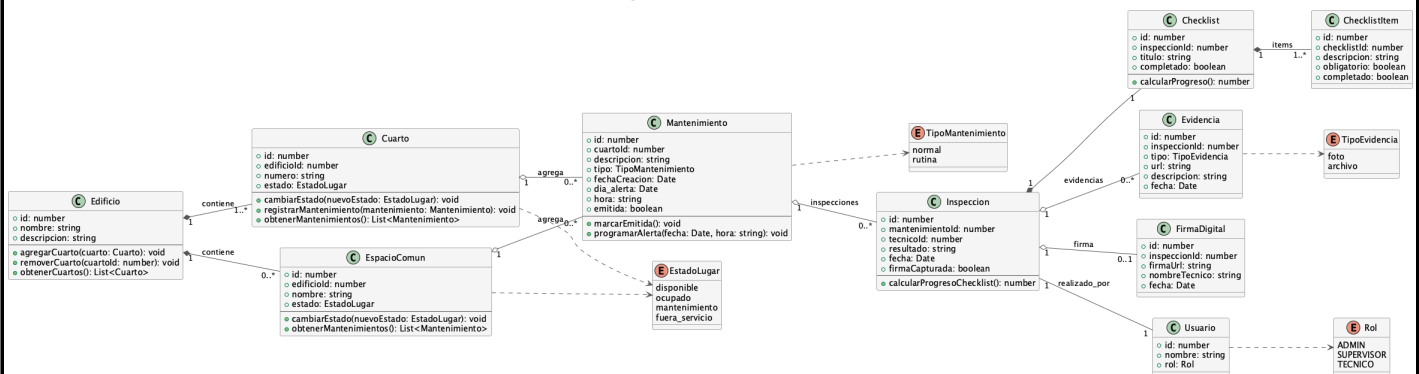


Diagrama de Clases:

- Clases principales: Edificio, Cuarto, Mantenimiento
- Atributos y métodos definidos para cada clase
- Relaciones: Composición (Edificio-Cuarto), Agregación (Cuarto-Mantenimiento)
- Multiplicidades: 1..* (un edificio tiene muchos cuartos)

Diagrama de Clases - SGSOM (JW Mantto)



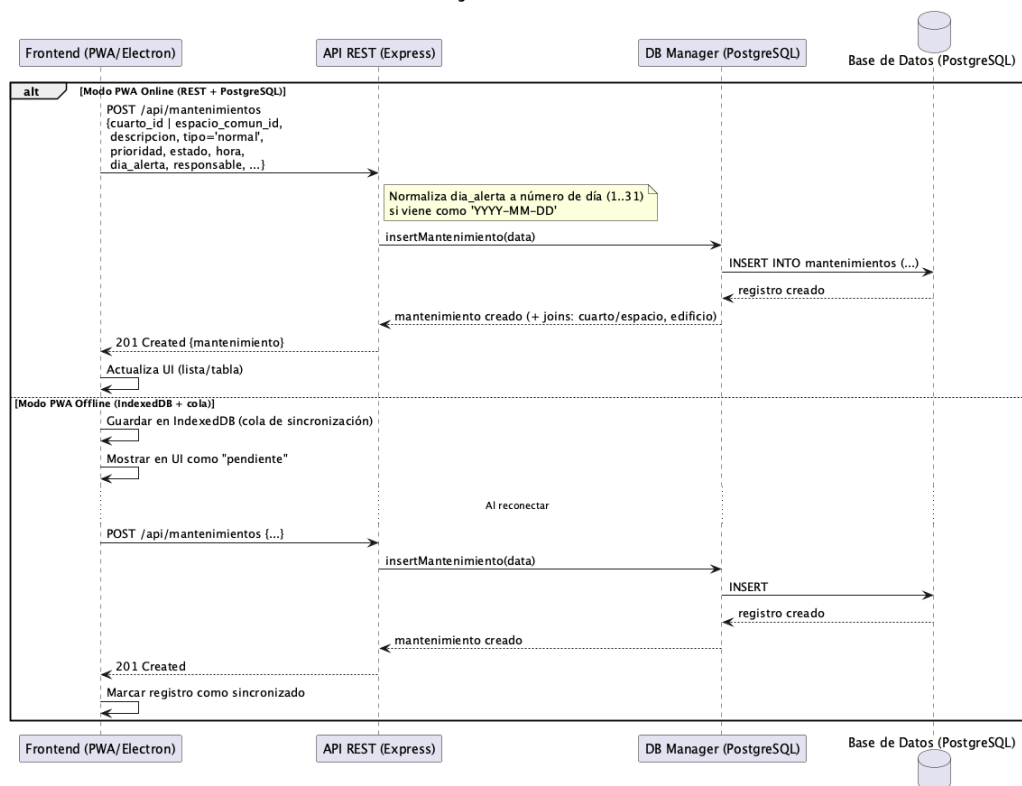
Diagramas de Secuencia:

- Secuencia: Registro de mantenimiento normal
- Secuencia: Cambio de estado de cuarto
- Secuencia: Emisión de alerta programada

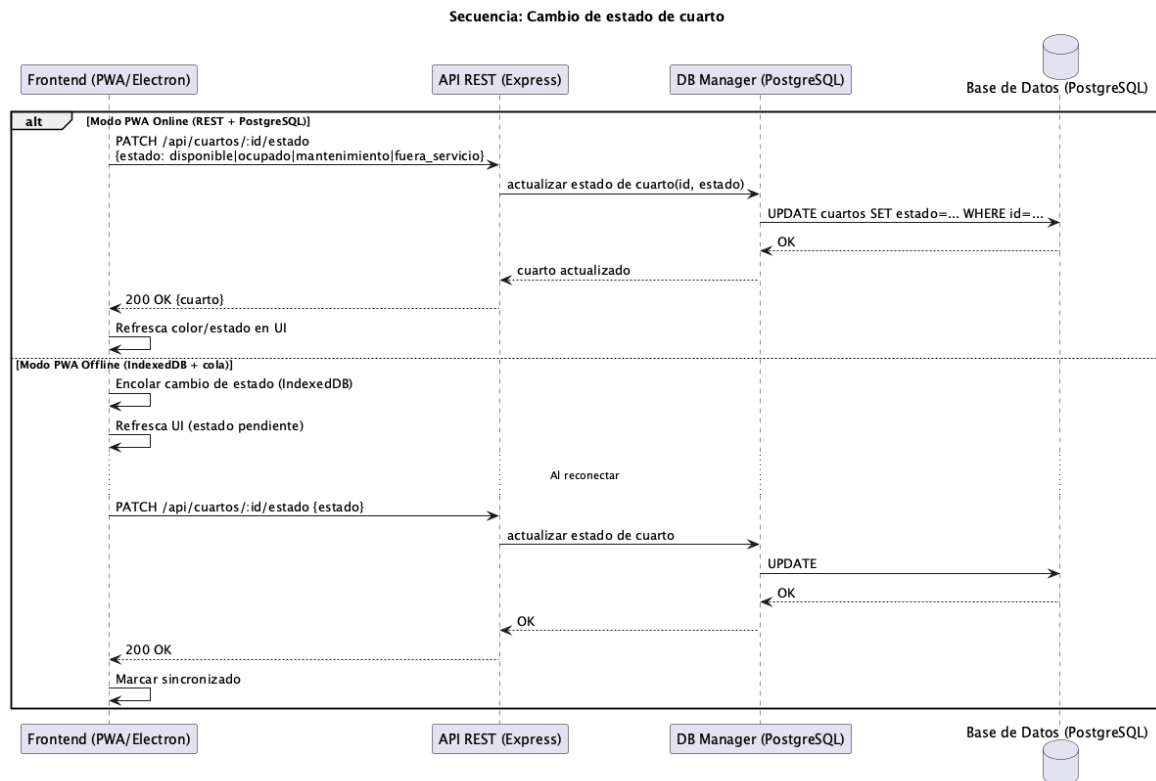
Tendrán interacción entre Frontend, API REST, Database Manager y Base de Datos

Registro de mantenimiento normal:

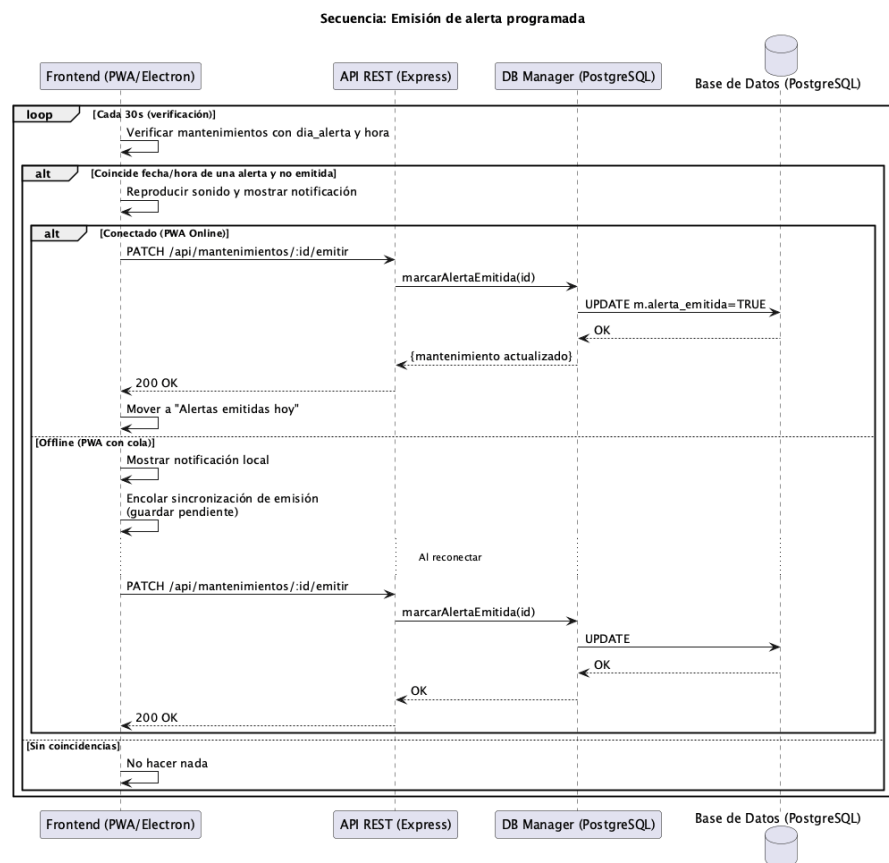
Secuencia: Registro de mantenimiento normal



Cambio de estado de cuarto:



Emisión de alerta programada:



Ahora definí todas las acciones que cada Usuario con su rol haría en el sistema:

Identificación de roles:

- **Personal de Mantenimiento:** Registrar y consultar mantenimientos, alertas e inspecciones y cambiar estados
- **Supervisor de Área:** Todas las funciones del personal de mantenimiento + asignar prioridades, aprobar completados de checklist, comprobar evidencias y firmas y acceso a reportes
- **Administrador del Sistema:** Gestión completa del sistema + configuración de usuarios

Matriz de permisos:

<i>Función</i>	<i>Mannt</i>	<i>Supervisor</i>	<i>Admin</i>
<i>Ver habitaciones, edificios y espacios</i>	✓	✓	✓
<i>Registrar mantenimiento</i>	✓	✓	✓
<i>Cambiar estado habitación y espacio</i>	✓	✓	✓
<i>Registrar inspecciones</i>	✓	✓	✓
<i>Configurar alertas</i>	✓	✓	✓
<i>Acceso al historial de mantenimientos</i>	✓	✓	✓
<i>Asignar prioridades</i>	X	✓	✓
<i>Editar mantenimiento</i>	X	✓	✓
<i>Eliminar mantenimiento</i>	X	✓	✓
<i>Aprobar completados de checklist, evidencias y firmas</i>	X	✓	✓

<i>Función</i>	<i>Mantt</i>	<i>Supervisor</i>	<i>Admin</i>
<i>Gestionar reportes</i>	X	✓	✓
<i>Gestionar edificios</i>	X	X	✓
<i>Gestionar habitaciones</i>	X	X	✓
<i>Gestionar espacios comunes</i>	X	X	✓
<i>Configuración de usuarios</i>	X	X	✓

Se agregarán logins y tokens de sesión para reforzar la seguridad del sistema.

Especificación de autenticación

- Sistema de login con usuario y contraseña (preparado para implementación futura)
- Tokens de sesión para mantener autenticación
- Cierre de sesión automático por inactividad (30 minutos)

Con las especificaciones del sistema, flujo de usos, permisos y acciones, podremos empezar a iniciar el espacio de trabajo con las tecnologías de desarrollo.

Configuración avanzada de Node.js

- Variables de entorno con dotenv (.env para configuración local)
- Scripts npm para desarrollo, producción y testing

Mi ejemplo en dotenv:

```
# =====  
# CONFIGURACIÓN DE POSTGRESQL  
# =====  
  
# Configuración para desarrollo local (macOS)  
DB_HOST=localhost  
DB_PORT=5432  
DB_NAME=jwmantto  
DB_USER=leonardocruz  
DB_PASSWORD=
```

Módulos de Node.js del proyecto

Dependencias:

Módulo	Versión	Descripción	Uso en el Proyecto
<i>express</i>	4.21.2	Framework web para Node.js	Servidor HTTP y API REST
<i>pg</i>	8.13.1	Cliente PostgreSQL para Node.js	Conexión a base de datos PostgreSQL
<i>cors</i>	2.8.5	Middleware para habilitar CORS	Permitir peticiones desde diferentes orígenes
<i>dotenv</i>	16.4.5	Carga variables de entorno desde .env	Configuración segura de credenciales

Como están indicadas en *package.json* se pueden instalar todas usando el comando:

```
# Instalar todas las dependencias
npm install
```

Ahora instalamos el gestor de base de datos que utilizaremos, será PostgreSQL y posteriormente cargaré datos de cuartos y edificios que me otorgaron anteriormente, con mantenimientos de prueba.

Instalación PostgreSQL

Instalación usando Homebrew en macos:

```
brew install postgresql
```

Comprobamos

```
psql --version
```

```
leonardocruz — -zsh — 80x24

Last login: Sun Nov  9 18:53:58 on ttys002
leonardocruz@MacBook-Air-de-leonardo ~ % psql --version
psql (PostgreSQL) 14.19 (Homebrew)
leonardocruz@MacBook-Air-de-leonardo ~ %
```

Consultas a la base de datos para ver cuartos, edificios y mantenimientos de prueba.

Listamos bases de datos para ver si está creada:

```
$ psql -l
List of databases
+-----+-----+-----+-----+-----+-----+
Name      | Owner   | Encoding | Collate | Ctype | Access privileges |
+-----+-----+-----+-----+-----+-----+
jwmantto  | leonardocruz | UTF8     | C       | C     |                    |
postgres | leonardocruz | UTF8     | C       | C     |                    |
template0 | leonardocruz | UTF8     | C       | C     | =c/leonardocruz +
          |          |          |          |          | leonardocruz=CtC/leonardocruz +
          |          |          |          |          | =c/leonardocruz +
          |          |          |          |          | leonardocruz=CtC/leonardocruz
template1 | leonardocruz | UTF8     | C       | C     |                    |
(4 rows)
```

```
$ psql -d jwmanitto -c "\dt"
List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | cuartos        | table | leonardocruz
public | edificios      | table | leonardocruz
public | mantenimientos | table | leonardocruz
(3 rows)
```

cleareq-9f ezrgqo qezclrbctou eqititcto iq unwelo 922) iq	ftwezrgamb Atimong ftwe zome cpelecfel A9Lltud(20) fexf zufe6el cpelecfel A9Lltud(T00)			CNBBEIL-TIMEZLWNB ,,qzaboutrpe,,c:cp9elcfel A9Lltud wof unff wof unff wof unff wexf9f9f,,cpelecfel-tq-zed,,:ledcf
colnuu	ylbe	colrefetou	unff9f9f	pe9f9f
\$ b2df -q jmw9f9f -c „/q cpelecfel,, trpe „b9f9f:c:cp9elcfel,,				

```
$ psql -d jwmantto -c "\d edificios"
```

Column	Type	Table "public.edificios"	Collation	Nullable	Default
id	integer			not null	nextval('edificios_id_seq'::reg
nombre	character varying(100)			not null	
descripcion	text				
created_at	timestamp without time zone				CURRENT_TIMESTAMP

```
$ psql -d jwmanitto -c "\d mantenimientos"
Table "public.mantenimientos"
  Column          | Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
id                | integer       |           | not null | nextval('mantenimientos_id_seq'::regclass)
cuarto_id         | integer       |           | not null | 
descripcion       | text          |           | not null | 
tipo              | character varying(50) |           |          | 'normal'::character varying
estado            | character varying(50) |           |          | 'pendiente'::character varying
fecha_creacion    | timestamp without time zone |           |          | CURRENT_TIMESTAMP
fecha_programada  | date          |           |          | 
hora              | time without time zone |           |          | 
dia_alerta        | integer       |           |          | 
alerta_emitida    | boolean       |           |          | false
usuario_creador   | character varying(100) |           |          | 'sistema'::character varying
notas             | text          |           |          | 
```

```
$ psql -d jwmannto -c "SELECT * FROM edificios ORDER BY id;"
```

id	nombre	descripcion	created_at
1	Alfa	Edificio Alfa	
2	Bravo	Edificio Bravo	
3	Charly	Edificio Charly	
4	Eco	Edificio Eco	
5	Fox	Edificio Fox	
6	Casa Maat	Casa Maat	

(6 rows)

Listamos algunos cuartos de cada edificio:

```
$ psql -d jwmantto -c "SELECT c.id, c.numero, e.nombre as edificio, c.estado FROM cuartos c JOIN edificios e ON c.edificio_id = e.id ORDER BY c.id LIMIT 10;"
```

id	numero	edificio	estado
1	A101	Alfa	disponible
2	A102	Alfa	disponible
3	A103	Alfa	disponible
4	A104	Alfa	disponible
5	A105	Alfa	disponible
6	A106	Alfa	disponible
7	S-A201	Alfa	disponible
8	A202	Alfa	disponible
9	A203	Alfa	disponible
10	A204	Alfa	disponible

(10 rows)

Vemos el total de cuartos por edificio:

```
$ psql -d jwmantto -c "SELECT e.nombre as edificio, COUNT(c.id) as total_cuartos FROM edificios e LEFT JOIN cuartos c ON e.id = c.edificio_id GROUP BY e.nombre ORDER BY total_cuartos DESC;"
```

edificio	total_cuartos
Bravo	57
Fox	56
Charly	55
Eco	49
Casa Maat	47
Alfa	37

(6 rows)

Revisamos el tipo de mantenimientos que tenemos en la tabla *mantenimientos*:

```
$ psql -d jwmantto -c "SELECT tipo, estado, COUNT(*) as cantidad FROM mantenimientos GROUP BY tipo, estado ORDER BY tipo, estado;"
```

tipo	estado	cantidad
normal	pendiente	8
rutina	pendiente	6

(2 rows)

Mostramos algunos mantenimientos:

```
$ psql -d jwmantto -c "SELECT m.id, c.numero as cuarto, e.nombre as edificio, m.tipo, m.estado, m.descripcion, m.fecha_creacion FROM mantenimientos m JOIN cuartos c ON m.cuarto_id = c.id JOIN edificios e ON c.edificio_id = e.id ORDER BY m.fecha_creacion DESC;"
```

id	cuarto	edificio	tipo	estado	descripcion	fecha_creacion
71	A102	Alfa	rutina	pendiente	nuevo	2025-10-30 03:24:49.453351
64	A102	Alfa	normal	pendiente	foo	2025-10-30 03:03:43.200841
61	A102	Alfa	normal	pendiente	asd	2025-10-30 02:59:33.40711
60	A102	Alfa	normal	pendiente	camas	2025-10-30 02:57:37.440849
58	A102	Alfa	rutina	pendiente	olis5	2025-10-30 02:57:37.440849

Por último, el usuario del gestor postgres:

```
$ psql -d jwmantto -c "SELECT current_user, current_database();"
current_user | current_database
-----+-----
leonardocruz | jwmantto
(1 row)
```

Git y control de versiones

Ahora, para llevar el control de versiones, cambios y novedades en el desarrollo, crearé el repositorio GIT:

Información del Repositorio Local:

- **Ubicación:** ** `\\Volumes/SSD/jwm\_mant\_cuartos`
- **Nombre del directorio:** `jwm\_mant\_cuartos`
- **Sistema de control de versiones:** Git

Repositorio Remoto (GitHub)

- **Plataforma:** GitHub
- **URL:** `git@github.com:leonardorey-coder/jwm_mantenimiento.git`
- **Propietario:** leonardorey-coder
- **Nombre del repositorio:** jwm_mantenimiento
- **Tipo de acceso:** SSH (git@github.com)

Comandos para iniciar el Git:

```
Inicializar el repositorio Git  
git init
```

Como ya tengo mi cuenta, se puede omitir este paso, pero es recomendable asignar el usuario:

```
Configurar el nombre de usuario (si es primera vez)  
git config --global user.name "leonardo"
```

Agregamos todos los archivos actuales para subir cambios:

```
Agregar todos los archivos al staging area  
git add .
```

Y realice el primer commit:

```
Hacer el primer commit  
git commit -m "Primer commit"
```

Ahora procedo a subir los cambios al repo remoto en GitHub. Primero lo enlazo con el repo:

```
Agregar el repositorio remoto  
git remote add origin git@github.com:leonardorey-coder/jwm_mantenimiento.git
```

Verifico

```
Verificar que se agregó correctamente  
git remote -v
```

Y subo al repositorio en la rama principal

```
Hacer el primer push al repositorio remoto  
git push -u origin main
```

Documentación Técnica

Comienzo el README del Proyecto según los avances y requerimientos.

Sistema de Gestión de Servicios Operativa de Mantenimiento (SGSOM) – JW Mantto

Sistema moderno de registro y gestión de mantenimiento de habitaciones para hoteles, construido como ****PWA (Progressive Web App) con Node.js/Express + PostgreSQL****. Funciona online y offline, con ****sincronización automática**** cuando se recupera la conexión.

> Arquitectura actualizada: PWA + PostgreSQL con soporte offline y sincronización diferida (cola de cambios en BD local del navegador).

Nombre del Proyecto

- Nombre completo: ****Sistema de Gestión de Servicios Operativa de Mantenimiento (SGSOM)****
- Nombre corto/alias: ****JW Mantto****
- Contexto: ****JW Marriott Resort & Spa – Estancia I****
- Entregable de esta implementación: ****Backend (API REST y Base de Datos) + PWA online/offline****

Características Principales

- ****Gestión de Habitaciones y Espacios Comunes****: Administra habitaciones y áreas comunes por edificios
- ****Mantenimientos****: Registro de mantenimientos normales y rutinas programadas
- ****Alertas Programadas****: Sistema de notificaciones automáticas
- ****Offline-First****: Operación 100% offline (datos y acciones quedan en cola)
- ****PWA****: Instalable en móviles y equipos de escritorio vía navegador
- ****Base de datos central****: PostgreSQL (nube/servidor)
- ****BD local (offline)****: IndexedDB (cola de operaciones para sincronizar)
- ****Sincronización****: Reintento automático al recuperar conectividad

Objetivo General

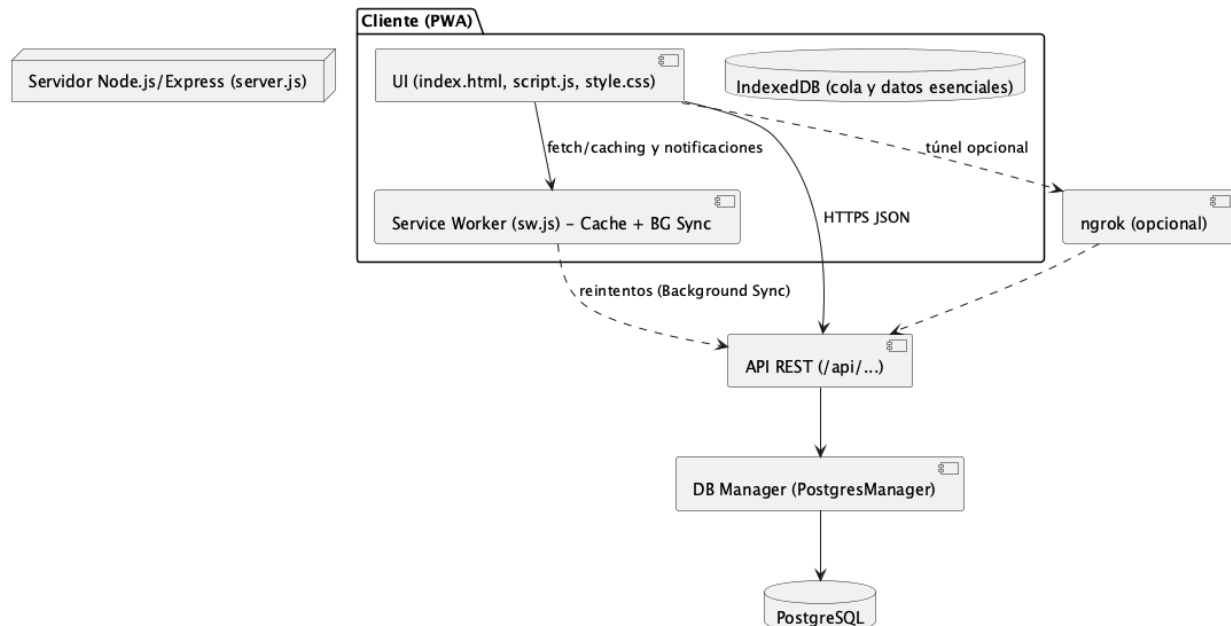
Diseñar e implementar un sistema web (PWA) para la gestión operativa de mantenimiento de habitaciones y edificios del hotel, con soporte online/offline, alertas programadas y sincronización confiable hacia una base de datos central en PostgreSQL.

Objetivos Específicos

- Proveer una interfaz web intuitiva y responsive para la gestión rápida de edificios, habitaciones, espacios comunes y mantenimientos.
- Implementar un CRUD completo para habitaciones, espacios comunes y mantenimientos con estados y tipos (normal/rutina).
- Incorporar un sistema de alertas programadas con notificaciones, sonido e historial.
- Operar en modo offline con IndexedDB y sincronización diferida al recuperar conexión.
- Centralizar datos en PostgreSQL para acceso multiusuario y escalabilidad.

Diagramas de arquitectura

Arquitectura General - SGSOM (PWA + Node/Express + PostgreSQL)



Documentación de API REST

Convenciones

- **Base URL local:** http://localhost:3001
- **Formato:** JSON
- **Errores:** { "error": string, "details"?: string }
- **Estados HTTP usados:** 200, 201, 400, 404, 500

Creo una api índice para poder cargar todas las api's y revisar si el servidor está iniciado correctamente y por completo:

```

/**
 * Configurador central de todas las rutas API
 */
const edificiosRouter = require('./edificios');
const cuartosRouter = require('./cuartos');
const mantenimientosRouter = require('./mantenimientos');

function setupApiRoutes(app, dbManager) {
  console.log('📁 Configurando rutas de API...');

  // Health check endpoint
  app.get('/api/health', (req, res) => {
    res.json({
      status: 'ok',
      timestamp: new Date().toISOString(),
      database: dbManager ? 'connected' : 'disconnected'
    });
  });
}

```

```
// Registrar routers por recurso
app.use('/api/edificios', edificiosRouter(dbManager));
app.use('/api/cuartos', cuartosRouter(dbManager));
app.use('/api/mantenimientos', mantenimientosRouter(dbManager));

console.log('Rutas de API configuradas');
}
module.exports = setupApiRoutes;
```

Archivo: api/mantenimientos.js:

Este módulo implementa todas las operaciones CRUD para mantenimientos.

```
const express = require('express');
const router = express.Router();

/**
 * Módulo de API para mantenimientos
 * @param {PostgresManager} dbManager - Gestor de base de datos
 */
module.exports = (dbManager) => {

  // =====
  // GET /api/mantenimientos
  // Obtener todos los mantenimientos
  // =====
  router.get('/', async (req, res) => {
    try {
      if (!dbManager) {
        return res.status(500).json({
          error: 'Base de datos no disponible'
        });
      }

      // Filtro opcional por cuarto
      const cuartoId = req.query.cuarto_id;

      // Llamar al gestor de base de datos
      const mantenimientos = await dbManager.getMantenimientos(cuartoId);

      // Devolver JSON
      res.json(mantenimientos);

    } catch (error) {
      console.error('Error al obtener mantenimientos:', error);
      res.status(500).json({
        error: 'Error al obtener mantenimientos',
        details: error.message
      });
    }
  });
};
```

```
// =====
// GET /api/mantenimientos/:id
// Obtener un mantenimiento específico
// =====
router.get('/:id', async (req, res) => {
  try {
    const { id } = req.params;

    if (!dbManager) {
      return res.status(500).json({
        error: 'Base de datos no disponible'
      });
    }

    const mantenimiento = await dbManager.getMantenimientoById(parseInt(id));

    if (!mantenimiento) {
      return res.status(404).json({
        error: 'Mantenimiento no encontrado'
      });
    }

    res.json(mantenimiento);

  } catch (error) {
    console.error('Error:', error);
    res.status(500).json({
      error: 'Error al obtener mantenimiento',
      details: error.message
    });
  }
});

// =====
// POST /api/mantenimientos
// Crear un nuevo mantenimiento
// =====
router.post('/', async (req, res) => {
  try {
    // 1. Extraer datos del body (ya parseado por express.json())
    const {
      cuarto_id,
      descripcion,
      tipo = 'normal',
      hora,
      dia_alerta
    } = req.body;

    console.log('📄 Creando mantenimiento:', {
      cuarto_id, descripcion, tipo, hora, dia_alerta
    });
  }
});
```

```
// 2. Validar campos obligatorios
if (!cuarto_id || !descripcion) {
  return res.status(400).json({
    error: 'Faltan campos obligatorios',

    required: ['cuarto_id', 'descripcion']
  });
}

// 3. Validar campos específicos de rutinas
if (tipo === 'rutina' && !hora) {
  return res.status(400).json({
    error: 'La hora es obligatoria para mantenimientos tipo rutina'
  });
}

if (!dbManager) {
  return res.status(500).json({
    error: 'Base de datos no disponible'
  });
}

// 4. Procesar dia_alerta (convertir fecha a número de día)
let diaAlertaNumero = null;
if (dia_alerta) {
  if (typeof dia_alerta === 'string' && dia_alerta.includes('-')) {
    // "2025-11-15" → extraer "15"
    const partes = dia_alerta.split('-');
    diaAlertaNumero = parseInt(partes[2]);
  } else {
    diaAlertaNumero = parseInt(dia_alerta);
  }
}

// 5. Preparar objeto de datos
const dataMantenimiento = {
  cuarto_id: parseInt(cuarto_id),
  descripcion,
  tipo,
  hora: hora || null,
  dia_alerta: diaAlertaNumero,
  fecha_solicitud: new Date().toISOString().split('T')[0],
  estado: 'pendiente'
};

// 6. Insertar en base de datos
const nuevoMantenimiento = await dbManager.insertMantenimiento(
  dataMantenimiento
);

console.log('✅ Mantenimiento creado:', nuevoMantenimiento);
```

```
// 7. Devolver el registro creado con código 201 (Created)
res.status(201).json(nuevoMantenimiento);

} catch (error) {
  console.error('❌ Error al crear mantenimiento:', error);
}

});

// =====
// PUT /api/mantenimientos/:id
// Actualizar un mantenimiento existente
// =====
router.put('/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const { descripcion, hora, dia_alerta, tipo, estado } = req.body;
    const mantenimientoId = parseInt(id);

    console.log('➡ Actualizando mantenimiento:', mantenimientoId);

    if (!dbManager) {
      return res.status(500).json({
        error: 'Base de datos no disponible'
      });
    }

    // Procesar dia_alerta
    let diaAlertaNumero = null;
    if (dia_alerta) {
      if (typeof dia_alerta === 'string' && dia_alerta.includes('-')) {
        const partes = dia_alerta.split('-');
        diaAlertaNumero = parseInt(partes[2]);
      } else {
        diaAlertaNumero = parseInt(dia_alerta);
      }
    }

    // Crear objeto con campos a actualizar
    const camposActualizar = {};
    if (descripcion !== undefined) camposActualizar.descripcion = descripcion;
    if (hora !== undefined) camposActualizar.hora = hora || null;
    if (diaAlertaNumero !== null) camposActualizar.dia_alerta = diaAlertaNumero;
    if (tipo !== undefined) camposActualizar.tipo = tipo;
    if (estado !== undefined) camposActualizar.estado = estado;

    // Actualizar en base de datos
    await dbManager.updateMantenimiento(mantenimientoId, camposActualizar);

    res.json({
      success: true,
      message: 'Mantenimiento actualizado correctamente'
    });
  }
});
```

```
    } catch (error) {
      console.error('✖ Error actualizando:', error);
      res.status(500).json({

    });

// =====
// PATCH /api/mantenimientos/:id/emitir
// Marcar una alerta como emitida
// =====
router.patch('/:id/emitir', async (req, res) => {
  try {
    const { id } = req.params;
    const mantenimientoId = parseInt(id);

    console.log('📢 Marcando alerta como emitida:', mantenimientoId);

    if (!dbManager) {
      return res.status(500).json({
        error: 'Base de datos no disponible'
      });
    }

    await dbManager.marcarAlertaEmitida(mantenimientoId);

    res.json({
      success: true,
      message: 'Alerta marcada como emitida'
    });

  } catch (error) {
    console.error('✖ Error:', error);
    res.status(500).json({
      success: false,
      message: 'Error interno del servidor',
      details: error.message
    });
  }
});

// =====
// DELETE /api/mantenimientos/:id
// Eliminar un mantenimiento
// =====
router.delete('/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const mantenimientoId = parseInt(id);

    console.log('🗑 Eliminando mantenimiento:', mantenimientoId);
```



```
    if (!dbManager) {
        return res.status(500).json({
            error: 'Base de datos no disponible'
        });
    }

    message: 'Mantenimiento eliminado correctamente'
});

} catch (error) {
    console.error('❌ Error eliminando:', error);
    res.status(500).json({
        success: false,
        message: 'Error interno del servidor',
        details: error.message
    });
}
});

return router;
};
```

Con eso finalizamos la implementación del CRUD completo a los mantenimientos.

Y podremos observar cómo otro script maneja la lógica con la conexión a la BD. Esta capa abstrae todas las operaciones de base de datos, proporcionando una interfaz limpia para las API's:

```
class PostgresManager {
    constructor() {
        this.pool = null; // Pool de conexiones
    }

    /**
     * Inicializar conexión a PostgreSQL
     */
    async initialize() {
        try {
            console.log('🔄 Inicializando PostgreSQL...');

            // Crear pool de conexiones
            this.pool = new Pool(dbConfig);

            // Manejar errores del pool
            this.pool.on('error', (err) => {
                console.error('❌ Error en pool PostgreSQL:', err);
            });

            // Probar conexión
            const client = await this.pool.connect();
            const result = await client.query('SELECT NOW()');
            console.log('✅ Conexión establecida:', result.rows[0].now);
        } catch (error) {
            console.error('❌ Error al inicializar PostgreSQL:', error);
        }
    }
}
```

```
        client.release();

        // Crear tablas si no existen
        await this.createTables();

        /**
         * Obtener todos los edificios
         * @returns {Promise<Array>} Array de edificios
         */
        async getEdificios() {
            const result = await this.pool.query(
                'SELECT * FROM edificios ORDER BY nombre'
            );
            return result.rows;
        }

        /**
         * Obtener todos los cuartos con información del edificio
         * @returns {Promise<Array>} Array de cuartos con join
         */
        async getCuartos() {
            const query = `
                SELECT c.*, e.nombre as edificio_nombre
                FROM cuartos c
                LEFT JOIN edificios e ON c.edificio_id = e.id
                ORDER BY e.nombre, c.numero
            `;
            const result = await this.pool.query(query);
            return result.rows;
        }

        /**
         * Obtener un cuarto específico por ID
         * @param {number} id - ID del cuarto
         * @returns {Promise<Object|null>} Cuarto o null si no existe
         */
        async getCuartoById(id) {
            const query = `
                SELECT c.*, e.nombre as edificio_nombre
                FROM cuartos c
                LEFT JOIN edificios e ON c.edificio_id = e.id
                WHERE c.id = $1
            `;
            const result = await this.pool.query(query, [id]);
            return result.rows[0] || null;
        }

        /**
         * Obtener mantenimientos, opcionalmente filtrados por cuarto
         * @param {number|null} cuartoId - ID del cuarto (opcional)
         * @returns {Promise<Array>} Array de mantenimientos
         */
        async getMantenimientos(cuartoId = null) {
```

```
let query = `
    SELECT m.*,
           c.numero as cuarto_numero,
           e.nombre as edificio_nombre
    FROM mantenimientos m
    LEFT JOIN cuartos c ON m.cuarto_id = c.id
    LEFT JOIN edificios e ON c.edificio_id = e.id
`;

let params = [];
if (cuartoId) {
    query += ' WHERE m.cuarto_id = $1';
    params.push(cuartoId);
}

query += ' ORDER BY m.fecha_creacion DESC';

const result = await this.pool.query(query, params);
return result.rows;
}
```

Endpoints API disponibles:

Método	Endpoint	Descripción
GET	/api/edificios	Obtener todos los edificios
GET	/api/cuartos	Obtener todos los cuartos
GET	/api/cuartos/:id	Obtener un cuarto específico
GET	/api/mantenimientos	Obtener todos los mantenimientos
POST	/api/mantenimientos	Crear un nuevo mantenimiento
PUT	/api/mantenimientos/:id	Actualizar un mantenimiento
DELETE	/api/mantenimientos/:id	Eliminar un mantenimiento
PATCH	/api/mantenimientos/:id/emitir	Marcar alerta como emitida

Filtrado y Búsqueda en Tiempo Real

Configuración de Eventos (Búsqueda en Tiempo Real):

Tengo un script para FrontEnd que, una vez cargados los cuartos, se puede filtrar por nombre (buscar) un cuarto sin tener que volver a consultar la BD:

```
/**
 * Configurar eventos de la interfaz
 * Archivo: app-loader.js (líneas 307-340)
 */
function configurarEventos() {
  // Configurar filtros
  const buscarCuarto = document.getElementById('buscarCuarto');
  const buscarAveria = document.getElementById('buscarAveria');
  const filtroEdificio = document.getElementById('filtroEdificio');

  // Búsqueda en tiempo real con evento 'input'
  if (buscarCuarto) buscarCuarto.addEventListener('input', filtrarCuartos);
  if (buscarAveria) buscarAveria.addEventListener('input', filtrarCuartos);
  if (filtroEdificio) filtroEdificio.addEventListener('change', filtrarCuartos);
}
```

Y mostramos la función *filtrarCuartos*:

```
/**
 * Filtrar cuartos según los criterios de búsqueda
 * Archivo: app-loader.js (líneas 533-555)
 */
function filtrarCuartos() {
  const buscarCuarto = document.getElementById('buscarCuarto').value.toLowerCase();
  const buscarAveria = document.getElementById('buscarAveria').value.toLowerCase();
  const filtroEdificio = document.getElementById('filtroEdificio').value;

  const cuartosFiltrados = cuartos.filter(cuarto => {
    // Filtro por nombre/número de cuarto
    const coincideNombre = (cuarto.nombre || cuarto.numero || '')
      .toString()
      .toLowerCase()
      .includes(buscarCuarto);

    // Filtro por avería en mantenimientos
    const coincideAveria = buscarAveria === '' ||
      (mantenimientos && mantenimientos.some(m =>
        m.cuarto_id === cuarto.id &&
        m.descripcion.toLowerCase().includes(buscarAveria)
      ));

    // Filtro por edificio
    const coincideEdificio = filtroEdificio === '' ||
      cuarto.edificio_id.toString() === filtroEdificio;

    return coincideNombre && coincideAveria && coincideEdificio;
  });

  mostrarCuartosFiltrados(cuartosFiltrados);
}
```

Filtrado y Búsqueda en Tiempo Real

Función Principal con Lazy Loading:

```
/**
 * Mostrar cuartos en la lista principal con lazy loading
 * Archivo: app-loader.js (líneas 345-441)
 */
function mostrarCuartos() {
  console.log('=== MOSTRANDO CUARTOS ===');

  const listaCuartos = document.getElementById('listaCuartos');
  if (!listaCuartos) {
    console.error('Elemento listaCuartos no encontrado');
    return;
  }

  console.log('Container encontrado:', listaCuartos);
  console.log('Cuartos disponibles:', cuartos ? cuartos.length : 0);

  if (!cuartos || cuartos.length === 0) {
    console.warn('No hay cuartos para mostrar');
    listaCuartos.innerHTML = '<li class="mensaje-no-cuartos">No hay cuartos registrados en el sistema.</li>';
    return;
  }

  console.log('Estructura del primer cuarto:', cuartos[0]);

  listaCuartos.innerHTML = '';
  let procesados = 0;

  // PASO 1: Crear cards vacías (placeholders)
  // Lazy loading: crear cards vacías y cargar contenido solo cuando entran al viewport
  cuartos.forEach((cuarto, index) => {
    try {
      const li = document.createElement('li');
      li.className = 'cuarto cuarto-lazy';
      li.id = `cuarto-${cuarto.id}`;

      // Guardar los datos necesarios en dataset para cargar luego
      li.dataset.nombreCuarto = cuarto.nombre || cuarto.numero || `Cuarto ${cuarto.id}`;
      li.dataset.edificioNombre = cuarto.edificio_nombre || `Edificio ${cuarto.edificio_id}`;
      li.dataset.descripcion = cuarto.descripcion || '';
      li.dataset.cuartoId = cuarto.id;
      li.dataset.edificioId = cuarto.edificio_id;
      li.dataset.index = index;

      // Card vacía (placeholder) - Carga inicial rápida
      li.innerHTML = `<div class="card-placeholder" style="height: 120px; background: #f3f3f3; border-radius: 10px;"></div>`;
    } catch (error) {
      console.error('Error al crear el elemento de la lista:', error);
    }
  });
}
```

```
        listaCuartos.appendChild(li);
        procesados++;
    } catch (error) {
        console.error(`Error procesando cuarto ${index}:`, error, cuarto);
    }
});

// PASO 2: IntersectionObserver para cargar el contenido real
// IntersectionObserver para cargar el contenido real cuando la card entra al viewport
if (window.cuartoObserver) {
    window.cuartoObserver.disconnect();
}

window.cuartoObserver = new IntersectionObserver((entries, observer) => {
    entries.forEach(entry => {
        // Cuando la card entra en el viewport
        if (entry.isIntersecting) {
            const li = entry.target;

            // Solo cargar si no se ha cargado antes
            if (!li.dataset.loaded) {
                // Obtener datos del dataset
                const cuartoId = parseInt(li.dataset.cuartoId);
                const nombreCuarto = li.dataset.nombreCuarto;
                const edificioNombre = li.dataset.edificioNombre;
                const descripcion = li.dataset.descripcion;
                const numMantenimientos = mantenimientos ?
                    mantenimientos.filter(m => m.cuarto_id === cuartoId).length : 0;

                // CARGAR CONTENIDO REAL
                li.innerHTML = `
                    <h2>${escapeHtml(nombreCuarto)}</h2>
                    <p class="edificio-cuarto">Edificio: ${escapeHtml(edificioNombre)}</p>
                    ${descripcion ? `<p>Última fecha: ${escapeHtml(descripcion)}</p>` : ''}
                    <p>Estado: <span class="estado-disponible">Disponible</span></p>
                    <p>Número de averías: <span id="contador-mantenimientos-
                    ${cuartoId}">${numMantenimientos}</span></p>
                    <button class="boton-toggle-mantenimientos"
                    onclick="toggleMantenimientos(${cuartoId}, this)">Mostrar Mantenimientos</button>
                    <ul class="lista-mantenimientos" id="mantenimientos-${cuartoId}"
                    style="display: none;">
                        ${generarMantenimientosHTMLSimple(mantenimientos.filter(m =>
                    m.cuarto_id === cuartoId))}
                    </ul>
                `;

                // Evento de selección
                li.addEventListener('click', function(e) {
                    if (!e.target.closest('button')) {
                        seleccionarCuarto(cuartoId);
                    }
                });
            }
        }
    });
});
```

```
// ✅ Marcar como cargado
li.dataset.loaded = '1';
li.classList.remove('cuarto-lazy');
li.style.opacity = '1';
li.style.transform = 'translateY(0)';
}

// ✅ Dejar de observar este elemento
observer.unobserve(li);
}
});
}, {
  rootMargin: '100px' // ✅ Cargar 100px antes de entrar al viewport
});

// ✅ PASO 3: Observar todas las cards lazy
document.querySelectorAll('.cuarto-lazy').forEach(li => {
  window.cuartoObserver.observe(li);
});

console.log(`Se procesaron ${procesados} cuartos de ${cuartos.length} total (lazy)`);
console.log('=== FIN MOSTRANDO CUARTOS ===');
}
```

Este script usa un prototipo de FrontEnd que muestra cada habitación como una tarjeta, donde evitamos cargar TODOS los cuartos del hotel usando un viewport; solo cargando los que están siendo visibles por el usuario en su ventana de navegador.

Control de Estados Dinámicos

Voy a empezar a definir las API's que consumirá el FrontEnd para cambiar el estado de las habitaciones y espacios comunes, siento los correspondientes:

- **Disponible:** Cuarto limpio y listo para ocupar
- **Ocupado:** Huésped hospedado actualmente
- **Mantenimiento:** En proceso de limpieza o reparación
- **Fuera de Servicio:** No disponible por remodelación o daños graves

Código de API que va a consumir el FrontEnd, donde también se define el Endpoint

```
/**
 * PATCH /api/cuartos/:id/estado
 * Actualizar el estado de un cuarto
 *
 * Estados permitidos:
 * - disponible: Cuarto limpio y listo para ocupar
 * - ocupado: Huésped hospedado actualmente
 * - mantenimiento: En proceso de limpieza o reparación
 * - fuera_servicio: No disponible por remodelación o daños graves
 */
router.patch('/:id/estado', async (req, res) => {
```

```
try {
  const { id } = req.params;
  const { estado } = req.body;

  console.log(`🔄 Cambiando estado del cuarto ${id} a: ${estado}`);

  // Validar que se envió el estado
  if (!estado) {
    return res.status(400).json({
      error: 'El campo "estado" es obligatorio',
      estadosPermitidos: ['disponible', 'ocupado', 'mantenimiento',
'fuera_servicio']
    });
  }

  if (!dbManager) {
    return res.status(500).json({ error: 'Base de datos no disponible' });
  }

  // Actualizar estado
  const cuartoActualizado = await dbManager.updateEstadoCuarto(
    parseInt(id),
    estado
  );

  console.log(`✅ Estado actualizado:`, cuartoActualizado);

  res.json({
    success: true,
    message: `Estado cambiado a "${estado}" correctamente`,
    cuarto: cuartoActualizado
  });
} catch (error) {
  console.error(`❌ Error actualizando estado:`, error);
  res.status(400).json({
    error: 'Error al actualizar estado',
    details: error.message
  });
}
});
```

Y el script que manejará la petición conectándolo con la BD:

```
/**
 * Actualizar el estado de un cuarto
 * @param {number} id - ID del cuarto
 * @param {string} nuevoEstado - Nuevo estado del cuarto
 * @returns {Promise<Object>} Cuarto actualizado
 */
async updateEstadoCuarto(id, nuevoEstado) {
  // Validar estados permitidos
  const estadosPermitidos = ['disponible', 'ocupado', 'mantenimiento', 'fuera_servicio'];
```



```
if (!estadosPermitidos.includes(nuevoEstado)) {
  throw new Error(`Estado no válido. Debe ser uno de: ${estadosPermitidos.join(',
')}}`);
}

const query = `
  UPDATE cuartos
  SET estado = $1,
      updated_at = CURRENT_TIMESTAMP
  WHERE id = $2
  RETURNING *
`;

const result = await this.pool.query(query, [nuevoEstado, id]);

if (result.rows.length === 0) {
  throw new Error('Cuarto no encontrado');
}

// Obtener el cuarto completo con información del edificio
return await this.getCuartoById(id);
}
```

Visualización con Códigos de Color

API que consumira el front para poder mandar peticion de asignamiento de color al estado:

```
/**
 * GET /api/cuartos/configuracion/estados
 * Obtener configuración de estados con colores e información
 *
 * Retorna la configuración completa de cada estado incluyendo:
 * - Color principal y secundario
 * - Icono
 * - Descripción
 * - Prioridad
 */
router.get('/configuracion/estados', async (req, res) => {
  try {
    console.log('👉 Obteniendo configuración de estados con colores');

    if (!dbManager) {
      return res.status(500).json({ error: 'Base de datos no disponible' });
    }

    const configuracion = dbManager.getConfiguracionEstados();

    console.log('✅ Configuración obtenida');

    res.json({
      success: true,
      estados: configuracion,
      version: '1.0',
    });
  } catch (error) {
    console.error('Error al obtener configuración de estados:', error);
    return res.status(500).json({ error: 'Error interno del servidor' });
  }
});
```

```
        descripcion: 'Configuración de estados de cuartos con paleta de colores'
    });

    } catch (error) {
        console.error('Error al obtener configuración:', error);
        res.status(500).json({
            error: 'Error al obtener configuración',
            details: error.message
        });
    }
});
```

Y la API que registra la petición en la BD:

```
/**
 * Obtener configuración de estados con colores
 * @returns {Object} Objeto con estados y sus propiedades (color, label, icono)
 */
getConfiguracionEstados() {
    return {
        disponible: {
            valor: 'disponible',
            label: 'Disponible',
            descripcion: 'Cuarto limpio y listo para ocupar',
            color: '#4CAF50', // Verde
            colorHex: '4CAF50',
            colorRgb: 'rgb(76, 175, 80)',
            colorSecundario: '#E8F5E9', // Verde claro para fondo
            icono: '●',
            prioridad: 1,
            disponibleParaReserva: true
        },
        ocupado: {
            valor: 'ocupado',
            label: 'Ocupado',
            descripcion: 'Huésped hospedado actualmente',
            color: '#2196F3', // Azul
            colorHex: '2196F3',
            colorRgb: 'rgb(33, 150, 243)',
            colorSecundario: '#E3F2FD', // Azul claro para fondo
            icono: '●',
            prioridad: 2,
            disponibleParaReserva: false
        },
        mantenimiento: {
            valor: 'mantenimiento',
            label: 'Mantenimiento',
            descripcion: 'En proceso de limpieza o reparación',
            color: '#FF9800', // Naranja
            colorHex: 'FF9800',
            colorRgb: 'rgb(255, 152, 0)',
            colorSecundario: '#FFF3E0', // Naranja claro para fondo
            icono: '●',
            prioridad: 3.
        }
    };
}
```

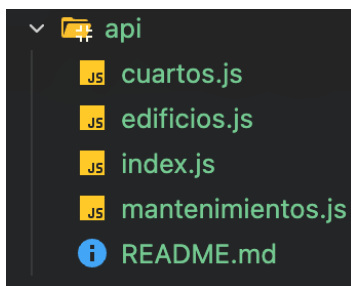
```

        disponibleParaReserva: false
    },
    fuera_servicio: {
        valor: 'fuera_servicio',
        label: 'Fuera de Servicio',
        descripcion: 'No disponible por remodelación o daños graves',
        color: '#616161', // Gris oscuro
        colorHex: '616161',
        colorRgb: 'rgb(97, 97, 97)',
        colorSecundario: '#F5F5F5', // Gris claro para fondo
        icono: '🕒',
        prioridad: 4,
        disponibleParaReserva: false
    }
};
}

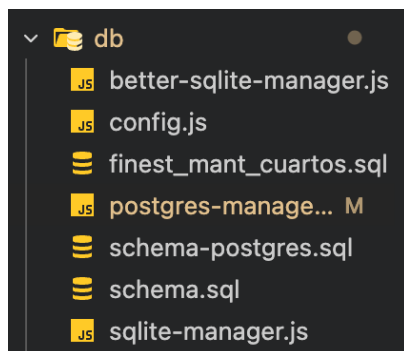
```

B. EVIDENCIAS FOTOGRAFICAS

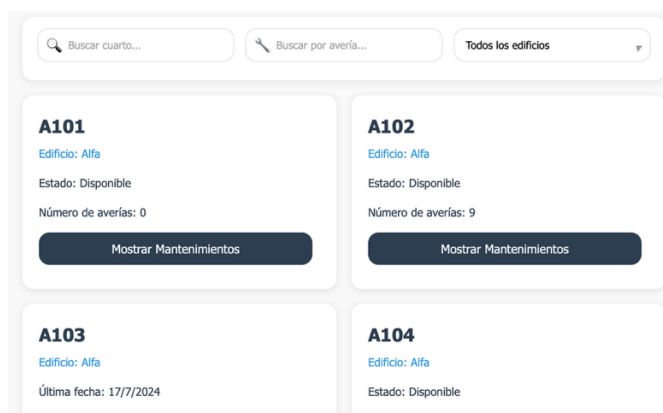
API's hasta el momento:



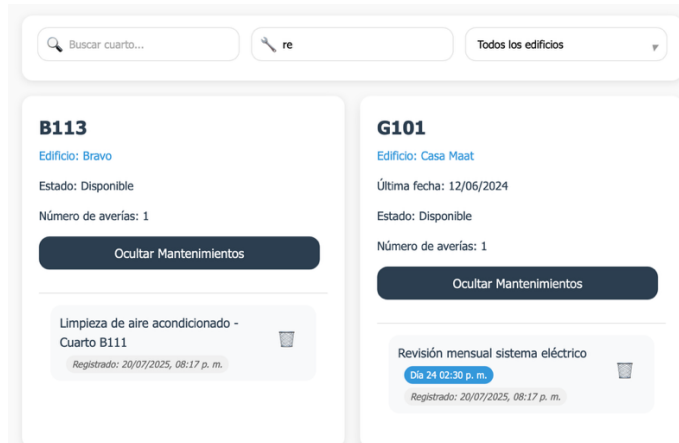
Junto con los scripts, API's de la BD y esquemas:



Prototipo de UI para probar la carga lenta (lazy loading)



Búsqueda por mantenimiento funcionando



The screenshot displays a web interface for searching maintenance records. At the top, there is a search bar with the placeholder text 'Buscar cuarto...', a filter button labeled 're', and a dropdown menu showing 'Todos los edificios'. Below the search bar, two maintenance records are shown side-by-side. The first record is for room B113, located in 'Edificio: Bravo', with a status of 'Disponible', 1 report, and a button to 'Ocultar Mantenimientos'. The second record is for room G101, located in 'Edificio: Casa Maat', with a status of 'Disponible', 1 report, and a button to 'Ocultar Mantenimientos'. Both records include a list of maintenance tasks with their dates and times. For B113, the task is 'Limpieza de aire acondicionado - Cuarto B111' recorded on 20/07/2025 at 08:17 p. m. For G101, the task is 'Revisión mensual sistema eléctrico' recorded on 20/07/2025 at 08:17 p. m., with a specific time slot of 'Día 24 02:30 p. m.' highlighted.

C. ETAPAS CUMPLIDAS

1. Diagramas Casos de Uso, Clases, Secuencia y de Arquitectura.
2. Definición de roles y permisos de los usuarios.
3. APIs REST funcionales (GET, POST, PUT, DELETE, PATCH)
4. Sistema de gestión de estados con 4 estados para habitaciones y espacios comunes
5. Búsqueda y filtrado en tiempo real
6. Lazy loading implementado
7. Códigos de color con 2 tonos de color por estado
8. Base de datos PostgreSQL configurada
9. Documentación técnica general del proyecto

Juan Leonardo Cruz Flores

NOMBRE Y FIRMA DEL ESTUDIANTE

NOMBRE Y FIRMA DEL ESTUDIANTE